Buffapoly — Rapport de conception technique

Adaptation 3D du Monopoly français avec Babylon.js 7 et TypeScript

Salima Mazaev · Naim Harib — Licence 3 MIAGE, Université Côte d'Azur
Concours Games on the Web 2026 — encadrants : Michel Buffa, Othman Mekouar

Table des matières

1. Introduction et choix du projet 3
1.1 Présentation de Buffapoly 3
1.2 Pourquoi le Monopoly plutôt qu'un autre jeu ? 3
1.3 Pourquoi Babylon.js plutôt que Three.js ? 3
2. Architecture logicielle 4
2.1 Quatre couches, dépendances vers le bas uniquement 4
2.2 Un EventBus typé comme contrat entre les couches 4
2.3 Machine à états du tour 5
3. Conception de la logique de jeu 5
3.1 Fonctions pures, Result type, immutabilité contrôlée 5
3.2 Plateau et cases 5
3.3 Règles notables 5
3.4 Cartes et Pattern Command 6
4. Rendu 3D et expérience visuelle 6
4.1 Plateau : un seul mesh, texture générée au runtime 6
4.2 Dés : simulation physique custom, sans Havok 6
4.3 Pions, bâtiments, caméra, éclairage 7
4.4 HoverCard3D : une carte Monopoly en 3D au survol 7
5. Synchronisation, IA et interface 7
5.1 Le problème central : règles instantanées vs animations longues 7
5.2 IA : heuristiques simples, pas de minimax 8
5.3 Interface DOM et audio procédural 8
6. Tests, difficultés et conclusion 8
6.1 Tests 8
6.2 Les trois difficultés principales 9
6.3 Ce qu'on referait, ce dont on est fiers 9
6.4 Perspectives 9

1. Introduction et choix du projet

1.1 Présentation de Buffapoly

Buffapoly est une adaptation 3D du Monopoly français jouable directement dans le navigateur, développée dans le cadre du concours *Games on the Web 2026*. Un joueur humain y affronte une à trois IA sur le plateau standard de 40 cases, avec l'ensemble des règles officielles : propriétés et monopoles, gares et compagnies, cartes Chance et Caisse de Communauté, prison, construction de maisons et d'hôtels, faillites avec transfert d'actifs. Le jeu est intégralement codé en TypeScript strict avec Babylon.js, sans asset 3D externe : le plateau, les pions, les dés et les bâtiments sont tous générés proceduralement à l'exécution.

1.2 Pourquoi le Monopoly plutôt qu'un autre jeu ?

Nous avons hésité avec plusieurs autres idées avant de trancher : un Puissance 4 en 3D, un Tetris revisité, ou des échecs avec pièces animées. Ces pistes présentaient toutes le même défaut à nos yeux : une fois l'effet 3D passé, le gameplay reste très simple et le projet se réduit à un exercice de rendu. Le Monopoly, au contraire, combine plusieurs axes intéressants en même temps.

D'abord, ses règles sont complexes et fortement interconnectées : un loyer dépend du monopole, un monopole débloque la construction, une construction modifie le loyer, une faillite redistribue les actifs selon le créancier. Ensuite, le plateau carré de 40 cases se prête particulièrement bien à une mise en scène 3D, contrairement à un Puissance 4 où la troisième dimension n'apporte rien au gameplay. Le fait que le jeu soit au tour par tour simplifie énormément la synchronisation des animations (on n'a jamais à gérer deux joueurs qui agissent en parallèle). Enfin, l'IA peut être rendue intéressante sans atteindre la complexité d'un moteur d'échecs : quelques heuristiques bien choisies suffisent pour un adversaire crédible.

1.3 Pourquoi Babylon.js plutôt que Three.js ?

Three.js était une alternative tout à fait crédible et probablement plus légère à l'embarquement. Nous avons finalement choisi Babylon.js 7 pour trois raisons concrètes. Babylon expose une API TypeScript native, sans dépendre de paquets @types externes parfois décalés du runtime. Surtout, beaucoup de briques dont nous avions besoin sont fournies d'usine : l'ArcRotateCamera avec ses contrôles souris/tactile, le DefaultRenderingPipeline (Bloom, ACES, FXAA), le SSAO2RenderingPipeline, le ShadowGenerator avec filtrage PCF, ou encore le PBRMaterial. Recréer ces briques sur Three.js aurait coûté un temps significatif pour un résultat visuel comparable. Enfin, la Playground et la documentation de Babylon nous ont fait gagner plusieurs journées de debug en phase d'exploration.

2. Architecture logicielle

2.1 Quatre couches, dépendances vers le bas uniquement

Le code est organisé en quatre couches strictement séparées, inspirées d'une Clean Architecture simplifiée. La règle que nous nous sommes imposée dès le début est que les imports ne vont que vers le bas : la couche haute connaît la couche basse, jamais l'inverse.

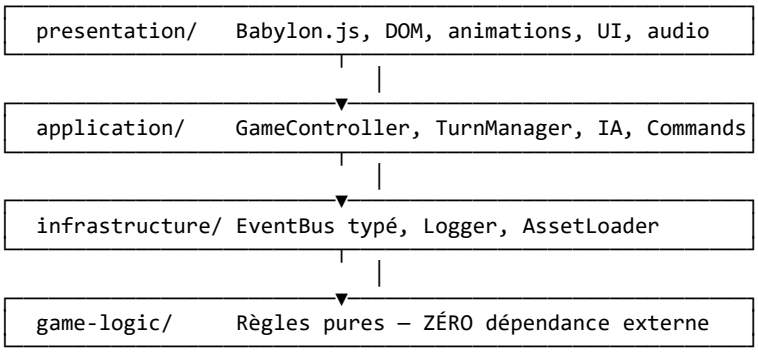

Figure 1 — Les quatre couches. La couche game-logic ne connaît ni Babylon.js ni le DOM.

Ce découpage nous a coûté un peu : on se retrouve avec beaucoup de petits fichiers, et il faut une discipline d'imports régulièrement vérifiée. En contrepartie, il a trois avantages qui ont payé. La couche game-logic est testable en Node pur, sans aucun DOM ni WebGL à mocker, ce qui nous a permis d'écrire plus de cent tests Vitest directement sur les règles. Un remplacement théorique de Babylon.js par Three.js ne toucherait que la couche presentation. Et surtout, nous avons pu travailler en parallèle à deux sans conflit : Salima sur game-logic et application, Naim sur presentation, l'EventBus servant de contrat entre les deux zones.

2.2 Un EventBus typé comme contrat entre les couches

Le cœur de ce découpage est un EventBus entièrement typé, défini dans infrastructure/event-bus.ts. Une interface GameEvents liste exhaustivement les événements du jeu avec leur payload exact, et TypeScript vérifie à la compilation que chaque emit et chaque on utilise le bon type.

```
interface GameEvents {
  'dice:rolled':        { values: [number, number]; isDouble: boolean };
  'pawn:moved':         { playerId: string; from: number; to: number;
                          steps: readonly number[] };
  'pawn:animation:complete': { playerId: string; to: number };
  'property:bought':    { playerId: string; squareIndex: number; price: number };
  'player:bankrupt':    { playerId: string };
  // ... une vingtaine d'événements au total
}
```

Cela élimine toute une classe de bugs : une typo dans un nom d'événement, un payload incomplet ou mal formé refusent de compiler. Quand le GameController émet dice:rolled, le DiceRenderer, le HudOverlay et l'AudioManager réagissent indépendamment sans se connaître les uns les autres.

2.3 Machine à états du tour

Le TurnManager pilote les phases d'un tour via une petite machine à états. Les transitions valides sont déclarées dans une table, et toute transition non autorisée lève une erreur, ce qui nous a permis de détecter très tôt des bugs d'orchestration.

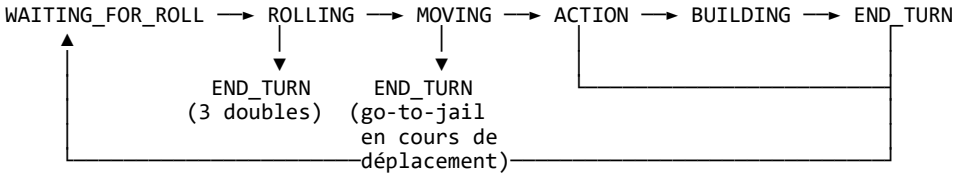

Figure 2 — Phases d'un tour. Les transitions alternatives gèrent 3 doubles et Go-to-Jail.

Un TurnPhase est donc une garde partagée entre l'état et le code : ACTION refuse un second lancer de dés, WAITING_FOR_ROLL refuse un achat, et ainsi de suite. Cela a surtout été utile quand nous avons branché l'IA : les actions de l'IA passent exactement par les mêmes méthodes que celles du joueur humain, et la machine à états nous garantit qu'elle ne peut pas tricher en sautant une phase.

3. Conception de la logique de jeu

3.1 Fonctions pures, Result type, immutabilité contrôlée

La couche game-logic est écrite presque exclusivement en fonctions pures qui prennent un GameState en entrée et retournent un résultat explicite. Quand une opération peut échouer (achat sans fonds, construction non équitable, sortie de prison sans carte), nous utilisons un type Result inspiré de Rust : soit { success: true, data }, soit { success: false, error: { code, message } }. L'appelant doit traiter les deux cas, ce qui supprime la tentation d'ignorer silencieusement une erreur.

3.2 Plateau et cases

Les 40 cases sont définies comme une union discriminée typée par un champ type. Chaque variante porte exactement les données dont elle a besoin : une PropertySquare a une couleur, un prix, un coût de maison et un tableau de six loyers (nu, 1 à 4 maisons, hôtel) ; une StationSquare n'a qu'un nom et un prix ; une TaxSquare porte son montant. TypeScript force à traiter tous les cas dans un switch, ce qui évite d'oublier une variante lors des évolutions.

3.3 Règles notables

Plusieurs règles officielles demandent plus de soin qu'on ne le croit au premier abord. Le loyer d'une propriété nue est doublé dès qu'on possède le monopole du groupe, mais seulement s'il n'y a aucun bâtiment ; sinon on utilise directement la ligne du tableau de loyers. Les gares suivent une progression 25 / 50 / 100 / 200 selon le nombre possédé. Les compagnies multiplient le dé par 4 ou par 10. La règle la plus subtile est celle de la construction uniforme : on ne peut construire sur une propriété d'un groupe que si elle a le moins de bâtiments du groupe, ou à égalité. Nous la vérifions par un minInGroup recalculé à chaque appel de canBuild. Nous respectons également les limites officielles de la banque : 32 maisons et 12 hôtels en circulation maximum, vérifiées par countBankUsage.

La prison est gérée en trois sorties possibles (double aux dés, amende de 50 €, carte « Sortez de prison ») avec un maximum de trois tours avant paiement forcé. Quand on sort par un double, on joue le déplacement du dé lancé mais on ne rejoue pas (nous avions initialement oublié cette nuance).

La faillite est probablement la règle la plus délicate à implémenter correctement, parce qu'elle a deux formes. Envers un autre joueur, le failli transfère toutes ses propriétés (avec bâtiments) et ses cartes « Sortez de prison » au créancier. Envers la banque, ses propriétés sont retirées du jeu et redeviennent achetables lors d'un futur passage, et ses cartes « Sortez de prison » sont rendues aux decks d'origine. La fonction transferBankruptAssets distingue ces deux cas via un argument creditorId nullable.

3.4 Cartes et Pattern Command

Les vingt cartes (dix Chance, dix Caisse de Communauté) sont définies en statique avec neuf types d'effets possibles, énumérés dans CardEffectType : MOVE_TO, MOVE_BY, MOVE_TO_NEAREST_STATION, GO_TO_JAIL, PAY, RECEIVE, COLLECT_FROM_ALL, PAY_REPAIRS (dépend du nombre de maisons et d'hôtels du joueur), et GET_OUT_OF_JAIL. La fonction applyCardEffect dispatche sur l'effet et retourne un CardEffectResult qui décrit proprement ce qui a changé (mouvement, variation de solde, envoi en prison).

Au-dessus de la logique pure, la couche application encapsule chaque action du joueur dans une Command sérialisable (RollDiceCommand, BuyPropertyCommand, BuildHouseCommand, EndTurnCommand). Chaque commande implémente une interface ICommand avec un type, un playerId, un timestamp et une méthode execute. Ce pattern nous ouvre la porte à la rejouabilité et à des tests déterministes en replay, même si nous n'exploitons ce potentiel que partiellement pour l'instant.

4. Rendu 3D et expérience visuelle

4.1 Plateau : un seul mesh, texture générée au runtime

Notre première implémentation utilisait 40 meshes séparés pour les 40 cases, ce qui produisait 40 draw calls rien que pour le plateau. Le résultat rendait correctement sur desktop mais chutait nettement sur mobile. Nous avons tout refait avec un seul plan texturé, la texture 2048 × 2048 étant générée à l'exécution via l'API Canvas 2D par le BoardTextureGenerator : fond, cases, bandeaux de couleur, noms, prix et cases spéciales sont dessinés une fois au démarrage, puis la DynamicTexture sert un unique draw call.

Piège concret rencontré : la face +Y du CreateBox de Babylon mappe les UV de +X vers −X, ce qui revient à afficher la texture en miroir horizontal. Notre premier résultat montrait tous les noms de rues inversés. La correction tient en deux lignes au début du dessin Canvas :

```
ctx.translate(TEX_SIZE, 0);
ctx.scale(-1, 1);  // compense le flip U de la face +Y du CreateBox
```

4.2 Dés : simulation physique custom, sans Havok

Plutôt que d'embarquer Havok ou Cannon uniquement pour deux dés, nous avons écrit une mini-simulation dans le DiceRenderer : gravité constante, rebonds avec coefficient, friction linéaire et angulaire, détection de repos quand la vitesse passe sous un seuil. Une fois le dé stabilisé, nous précalculons les 24 orientations plates possibles (six faces multipliées par quatre rotations autour de Y) et nous faisons un Slerp vers la plus proche qui affiche la valeur attendue. Le résultat a un aspect naturel sans dépendance lourde.

Bug anti-parallèle (deux jours perdus) : notre fonction quatFromUnitVectors gérait correctement le cas général via un cross product, mais le cas anti-parallèle (dot ≈ −1) construisait un quaternion en passant les composantes du vecteur perpendiculaire à FromEulerAngles, ce qui est mathématiquement incohérent (x, y, z sont des coordonnées, pas des angles). Résultat : les dés tombaient proprement mais affichaient systématiquement la mauvaise face. La correction est d'utiliser la vraie fonction prévue pour ça :

```
// AVANT (faux) :
// return Quaternion.FromEulerAngles(perp.x, perp.y, perp.z);
// APRÈS :
return Quaternion.RotationAxis(perp, Math.PI);
```

4.3 Pions, bâtiments, caméra, éclairage

Les quatre pions (chapeau, fer, voiture, dé) sont construits proceduralement à partir de primitives, avec chacun une couleur distinctive. Leur animation de déplacement utilise une interpolation case par case avec un petit arc parabolique en Y pour simuler un saut. Les maisons et hôtels utilisent des InstancedMesh sur des templates cachés, ce qui divise d'autant les draw calls quand plusieurs joueurs construisent en même temps.

La caméra est un point délicat. Plutôt que d'utiliser des Animation Babylon pour les transitions cinématiques, nous faisons un lerp frame-par-frame dans onBeforeRender vers un « goal » courant. Ce choix nous semblait moins élégant au départ, mais il autorise des interruptions instantanées : quand le joueur bouge pendant qu'on zoome sur une case, la caméra change de cible immédiatement au lieu d'attendre la fin de l'animation Babylon. Un ghost highlight semi-transparent pulsant signale visuellement la case d'un achat en cours.

L'éclairage est en trois points : une DirectionalLight chaude figurant un soleil à 35° (qui porte aussi les ombres PCF Soft), une HemisphericLight en fill ambient doux, et une PointLight mobile qui illumine la case active. Le pipeline de post-processing ajoute SSAO2 pour l'occlusion ambiante, Bloom léger, tonemapping ACES, FXAA et une vignette subtile.

4.4 HoverCard3D : une carte Monopoly en 3D au survol

Quand le curseur passe sur la case où se trouve le pion humain, une carte Monopoly 3D apparaît en lévitation au-dessus du plateau, orientée face caméra, avec le loyer détaillé, les coûts de construction et le propriétaire éventuel. Toute la carte est dessinée sur une DynamicTexture appliquée à un Plane. La difficulté a été de synchroniser la pick zone avec le pion : le GameState place le joueur instantanément, mais l'animation du pion prend plusieurs centaines de millisecondes. Nous activons la pick zone seulement après un délai égal au nombre de cases traversées multiplié par 320 ms (durée de saut d'une case), pour éviter que la carte de la destination apparaisse pendant que le pion est encore en route.

5. Synchronisation, IA et interface

5.1 Le problème central : règles instantanées vs animations longues

Le problème le plus délicat du projet a été la synchronisation entre les règles de jeu et les animations 3D. Concrètement, quand le joueur lance les dés, la position change instantanément dans le GameState, mais l'animation du pion prend plusieurs centaines de millisecondes. Sans précaution, on se retrouve à proposer l'achat d'une propriété alors que le pion est encore en train de sauter de case en case, ce qui casse complètement l'illusion.

Nous avons résolu le problème en introduisant deux événements dédiés, dice:animation:complete et pawn:animation:complete, que le GameController attend explicitement avant de résoudre la case atteinte. Le flux complet d'un tour ressemble donc à ceci :

```
1. joueur clique « Lancer »
2. GameController.handleRollDice() → rollDice() met à jour le state
3. emit dice:rolled                → DiceRenderer lance l'animation
4. waitingForDiceAnimation = true  → on ATTEND
5. dice:animation:complete reçu    → continueAfterDiceAnimation()
6. movePlayer(state)               → state modifié instantanément
7. emit pawn:moved                 → PawnController saute case/case
8. scheduleAfterPawnAnimation(...) → on ATTEND à nouveau
9. pawn:animation:complete reçu    → resolveSquare() : achat / loyer / carte
```

Figure 3 — Orchestration d'un tour. Deux points d'attente sérialisent règles et rendu.

5.2 IA : heuristiques simples, pas de minimax

Nous avons volontairement écarté l'idée d'un minimax pour l'IA. Le facteur de branchement d'un Monopoly est trop élevé (chaque tour implique un tirage de dés, un choix potentiel d'achat, des décisions de construction, des cartes aléatoires), et surtout le jeu n'est pas déterministe, donc le principal intérêt du minimax (projeter loin dans le futur) est largement perdu. Nous avons préféré un système d'heuristiques simples mais explicables.

Pour un achat, l'IA part d'un score de 50 et ajuste : +25 si la propriété complète un monopole, +15 si elle possède déjà un membre du groupe, +10 si la case est à fort trafic (les statistiques montrent que les cases 15 à 25 sont les plus fréquentées à cause du retour de prison), –20 si le solde post-achat passe sous 300 €, encore –20 sous 100 €. Au-dessus de 60 points, elle achète. Pour la construction, les groupes orange et rouge sont prioritaires (ils ont le meilleur rapport trafic/loyer), avec un solde plancher de 500 € conservé en trésorerie. Ce n'est pas optimal, mais l'IA joue visiblement « bien » aux yeux d'un joueur humain.

5.3 Interface DOM et audio procédural

Nous avons choisi le DOM plutôt que l'AdvancedDynamicTexture de Babylon pour toute l'interface 2D (HUD, panneau d'actions, notifications, panneau de propriétés, écran de fin de partie). Le DOM rend mieux le texte, les animations CSS sont gratuites, et surtout cela n'ajoute aucun draw call. La seule exception est la HoverCard3D, qui doit vraiment s'intégrer en 3D dans la scène et pour laquelle la logique BJS est plus pertinente.

L'audio est entièrement procédural via la Web Audio API : aucun fichier .mp3 ou .ogg embarqué. Un AudioManager génère à la volée les sons voulus avec des OscillatorNode (onde sinus, carrée, dent de scie selon l'effet), enveloppes de gain exponentielles et filtres bandpass pour les bruits de dés. Chaque événement du bus déclenche le son correspondant. Le poids total est quasi nul et le résultat est suffisant pour accompagner le gameplay.

6. Tests, difficultés et conclusion

6.1 Tests

Nous avons écrit plus de cent tests unitaires avec Vitest, organisés par couche. La couverture dépasse 75 % sur les trois couches non-rendu (game-logic, application, infrastructure), seuil que nous avons fixé dans la configuration de coverage V8 pour faire échouer la CI en dessous. La couche presentation n'est pas testée unitairement : il faudrait mocker le DOM et un contexte WebGL, pour un rapport coût/bénéfice que nous n'avons pas jugé intéressant. GitHub Actions exécute à chaque push typecheck, lint, prettier check, tests avec coverage, puis build.

6.2 Les trois difficultés principales

Synchronisation animations/règles. La première implémentation résolvait les cases synchroniquement : achat proposé avant même que le pion bouge. Solution : deux événements d'attente (dice:animation:complete, pawn:animation:complete) sérialisant les phases. Décrit en section 5.1.

Miroir UV de la face +Y. La texture du plateau s'affichait avec tout le texte inversé à cause du mapping UV de CreateBox. Compensé par un scale(−1, 1) dans le Canvas avant le dessin.

Cas anti-parallèle des dés. quatFromUnitVectors construisait un quaternion avec FromEulerAngles(perp.x, perp.y, perp.z), utilisant des coordonnées comme des angles. Les dés affichaient la mauvaise face. Remplacé par Quaternion.RotationAxis(perp, π).

6.3 Ce qu'on referait, ce dont on est fiers

Avec du recul, nous aurions gagné du temps à introduire pawn:animation:complete dès le début plutôt qu'à le rajouter après avoir rencontré le bug, et nous aurions écrit les tests du TurnManager plus tôt (ils ont été différés alors qu'ils auraient révélé plusieurs bugs d'orchestration).

Nous sommes particulièrement fiers de la HoverCard3D, qui donne un côté très « jeu physique » au projet sans coût de performance notable, et de la qualité du rendu obtenu entièrement en procédural : un nouvel arrivant sur le code ne trouve aucun .glb, aucune texture PNG, aucun .mp3, et pourtant le résultat se tient visuellement et sonore.

6.4 Perspectives

Plusieurs extensions nous intéresseraient : un mode multi-joueur en réseau (le découpage en Commands sérialisables a été pensé en partie pour cela), le remplacement des pions procéduraux par de vrais .glb téléchargés à la demande, une physique Havok activable en option pour les dés si l'utilisateur accepte le coût, et une IA plus fine mélangeant les heuristiques actuelles à un léger rollout pour les situations de fin de partie.

Nous remercions Michel Buffa et Othman Mekouar pour leur encadrement et pour la mise en place du concours Games on the Web, qui a été pour nous une excellente occasion de pousser un projet technique de bout en bout.